Labo 17 — Labo pratique : packager, livrer, annuler

Objectif : installer Helm, découvrir que k3s l'utilise déjà, packager la stack complète (web, api, db) en un chart, l'installer, la mettre à jour, revenir en arrière, rendre la même chose avec Kustomize, puis faire tirer le cluster depuis le registry du labo 10, par tag puis par digest. Et, pour finir la série, nettoyer tout.

Prérequis — Labo 16 terminé : k3s ; `localhost/api:1.0` et `localhost/api:1.1` importées dans containerd (labo 12) ; l'image `registry:2` et `api:1.1` dans Podman (labos 10 et 12).

Fichiers fournis (`files/`) - `stack/` — le chart Helm : `Chart.yaml`, `values.yaml`, `values-prod.yaml`, `templates/`, `files/`. - `kustomize/base/` et `kustomize/overlays/prod/` — la même API en base + overlay Kustomize.

Étape 1 — Installer Helm, et le voir déjà à l'œuvre

```
mkdir -p ~/labo-docker/17 && cd ~/labo-docker/17 && cp -r <chemin-du-labo>/files/* .
curl -fsSL https://get.helm.sh/helm-v4.3.1-linux-amd64.tar.gz | tar xz -C /tmp
sudo install /tmp/linux-amd64/helm /usr/local/bin/helm
helm version --short
helm list -A
```

Observez `v4.3.1+gc0d08d0`, puis `Error: kubernetes cluster unreachable: Get "http://localhost:8080/version": dial tcp [::1]:8080: connect: connection refused`.

```
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
helm list -A
kubectl get helmcharts -n kube-system
```

Observez deux releases dans `kube-system` : `traefik` et `traefik-crd`, chart `traefik-34.2.1+up34.2.0`, `deployed` depuis l'installation de k3s ; et les objets `HelmChart` qui les ont créées.

Explication. Helm est un simple client : sans `KUBECONFIG`, il ne sait pas où est le cluster. Et l'Ingress controller que vous utilisez depuis le labo 13 est un chart Helm installé par k3s.

> **→ WINDOWS / WSL**
> Ajoutez `export KUBECONFIG=/etc/rancher/k3s/k3s.yaml` à votre `~/.bashrc` : Helm, mais aussi k9s ou Lens installés côté Windows, en ont besoin. Le `kubectl` de k3s, lui, trouve le fichier seul.

Étape 2 — Lire le chart, le rendre, l'installer

```
ls -R stack | head -20
helm lint stack
helm template stack ./stack | grep -E '^kind:|^  name:' | paste - - | head -12
helm template stack ./stack --set api.replicas=5 | grep -m1 'replicas:'
```

Observez les cinq dossiers et fichiers du chart ; `1 chart(s) linted, 0 chart(s) failed` (un `[INFO] icon is recommended` sans gravité) ; onze objets rendus (`Secret db-secret`, trois `ConfigMap`, trois `Service`, deux `Deployment`, le `StatefulSet db`, l'`Ingress web`) ; et `replicas: 5` dès qu'on surcharge une valeur.

```
helm install stack ./stack -n lab17 --create-namespace
kubectl config set-context --current --namespace=lab17
sleep 25 ; kubectl get pods
helm list
curl -s localhost/api/ ; echo
```

Observez (un avertissement `SessionAffinity is ignored for headless services` est sans conséquence) `STATUS: deployed`, `REVISION: 1`, puis le texte de `NOTES.txt` avec les valeurs rendues ; quatre Pods `Running` (`api` ×2, `db-0`, `web`) ; `stack lab17 1 … deployed stack-0.1.0 1.0` ; et, via l'Ingress et le proxy nginx, `{"message":"bonjour depuis Helm","profile":"dev"}`.

Explication. Une commande a créé onze objets à partir d'un dossier et d'un fichier de valeurs. `helm template` les montre, `helm lint` les vérifie, `helm install` les applique et numérote la revision.

Étape 3 — Mettre à jour, consulter, revenir en arrière

```
helm upgrade stack ./stack --reuse-values --set api.message="bonjour depuis la revision 2" --
set api.replicas=3
kubectl rollout status deployment/api --timeout=90s
curl -s localhost/api/ ; echo
helm history stack
helm get values stack
```

Observez `Release "stack" has been upgraded. Happy Helming!` ; un rolling update de l'API ; le nouveau message dans la réponse ; deux revisions (`1 superseded`, `2 deployed … Upgrade complete`) ; et dans `USER-SUPPLIED VALUES`, exactement vos deux `--set`.

```
helm rollback stack 1
kubectl rollout status deployment/api --timeout=90s ; sleep 5
curl -s localhost/api/ ; echo
kubectl get pods -l app=api
helm history stack
```

Observez `Rollback was a success!` ; le message d'origine de retour ; deux Pods `api` (plus trois) ; et une **troisième** revision `deployed`, décrite `Rollback to 1` : les deux précédentes sont `superseded`.

Explication. Changer une valeur de ConfigMap a redéployé l'API grâce à l'annotation `checksum/config`. Le rollback n'efface pas l'histoire : il la prolonge d'une revision identique à la première.

Étape 4 — Un fichier de valeurs par environnement

```
cat stack/values-prod.yaml
helm template stack ./stack -f stack/values-prod.yaml | grep -E 'replicas:|APP_PROFILE|memory'
helm upgrade stack ./stack -f stack/values-prod.yaml --dry-run 2>&1 | grep -c 'kind:'
```

Observez un fichier qui ne contient **que les différences** avec `values.yaml` ; dans le rendu, `APP_PROFILE: "prod"`, `replicas: 3` pour l'API, `memory: 512Mi` / `128Mi`, `replicas: 2` pour le web ; et un `--dry-run` qui rend onze objets sans rien toucher.

Explication. Les templates ne changent jamais d'un environnement à l'autre ; seul le fichier de valeurs passé avec `-f` change. C'est le `compose.prod.yaml` du labo 09, pour un cluster.

Étape 5 — La même chose avec Kustomize

```
cat kustomize/overlays/prod/kustomization.yaml
kubectl kustomize kustomize/overlays/prod | grep -E
'^kind:|namespace:|replicas:|image:|env:|memory:'
kubectl apply -k kustomize/overlays/prod
sleep 20 ; kubectl get pods -n lab17-prod -o custom-
columns=NAME:.metadata.name,IMAGE:.spec.containers[0].image,ENV:.metadata.labels.env
kubectl delete -k kustomize/overlays/prod
```

Observez un overlay qui référence `../../base` et déclare `namespace`, `labels`, `replicas`, `images`, `patches` ; un rendu où la base, intacte, est devenue un Deployment à `replicas: 3`, image `localhost/api:1.1`, label `env: prod` et ressources patchées, dans `lab17-prod` ; trois Pods conformes ; puis la suppression du namespace et de son contenu.

Explication. Aucun template : des YAML purs, et un overlay qui dit ce qui change. `kubectl` sait rendre (`kustomize`) et appliquer (`-k`) sans rien installer.

Étape 6 — Tirer depuis le registry, par tag puis par digest

Côté Podman (labo 10) : relancez le registry et poussez `api:1.1` en notant son digest.

```
podman run -d --name registry -p 5000:5000 docker.io/library/registry:2 ; sleep 2
podman tag api:1.1 localhost:5000/api:1.1
podman push --tls-verify=false --digestfile digest.txt localhost:5000/api:1.1 2>&1 | tail -1
cat digest.txt ; echo
podman image inspect api:1.1 --format '{{.Digest}}'
```

Observez `Writing manifest to image destination`, un digest `sha256:b82020cb…` dans `digest.txt`, et un digest **différent** (`sha256:e1b2ba1e…`) pour l'image locale : le push a recompressé les couches ; celui du registry est le seul qui compte.

```
curl -s localhost:5000/v2/api/tags/list ; echo
helm upgrade stack ./stack --reuse-values --set api.image.repository=localhost:5000/api --set
api.image.tag=1.1
kubectl rollout status deployment/api --timeout=90s
kubectl get events --field-selector reason=Pulled | grep -m1 5000 | cut -c1-150
kubectl get pods -l app=api -o jsonpath='{.items[0].status.containerStatuses[0].imageID}' ;
echo
```

Observez `{"name":"api","tags":["1.1"]}` ; un rolling update ; `Successfully pulled image "localhost:5000/api:1.1" in 92ms` (vos valeurs diffèrent) — k3s a tiré depuis le registry, en HTTP, sans `registries.yaml` ; et `imageID` = `localhost:5000/api@sha256:b82020cb…` : le même digest que `digest.txt`.

```
DIGEST=$(kubectl get pods -l app=api -o
jsonpath='{.items[0].status.containerStatuses[0].imageID}' | cut -d@ -f2) ; echo $DIGEST
helm upgrade stack ./stack --reuse-values --set api.image.digest=$DIGEST
kubectl rollout status deployment/api --timeout=90s
kubectl get deployment api -o jsonpath='{.spec.template.spec.containers[0].image}' ; echo
helm history stack | cut -c1-110
```

Observez un Deployment dont l'image est maintenant `localhost:5000/api@sha256:b82020cb…` : plus de tag, une empreinte ; et cinq revisions dans l'historique.

Explication. Le tag `1.1` pourrait être réécrit demain ; le digest, jamais. La CI pousse avec un tag immuable et note le digest ; la production épingle le digest.

> **→ WINDOWS / WSL**
> Sur WSL, Podman et k3s sont sur la même machine : `localhost:5000` est le même registry pour les deux, et containerd accepte `localhost` en clair. Pour un registry sur une autre machine ou avec identifiants : `/etc/rancher/k3s/registries.yaml`, puis `sudo systemctl restart k3s`.

Nettoyage — et fin de la série

```
helm uninstall stack ; sleep 15
kubectl get all,pvc
kubectl delete namespace lab17 ; kubectl config set-context --current --namespace=default
helm list -A
podman rm -f -t 0 -v registry ; podman rmi localhost:5000/api:1.1
```

Observez après `uninstall`, plus aucun Pod, Service ni Deployment… mais la PVC `data-db-0` toujours `Bound` : Helm ne supprime pas ce que le StatefulSet a créé. La suppression du namespace l'emporte. `helm list -A` ne montre plus que Traefik.

Pour retirer **tout** k3s de votre WSL, quand vous aurez fini (irréversible, supprime aussi les volumes `local-path`) :

```
sudo /usr/local/bin/k3s-uninstall.sh
```

Les images Podman (`api:1.0`, `api:1.1`, `web:1.0`, `postgres`, `nginx`) et le volume `stack_pgdata` du labo 09 restent à vous : `podman images`, `podman volume ls`, et le labo 10 pour faire le tri.

Ce que vous devez pouvoir affirmer maintenant

- `helm` sans `KUBECONFIG` cherche `localhost:8080` ; avec, `helm list -A` montre Traefik installé par k3s.
- Chart `stack/` : `lint`, `template` (onze objets), `install` (revision 1, `NOTES.txt`), `curl localhost/api/`.
- `upgrade --set` → revision 2 et nouveau message ; `rollback 1` → revision 3 `Rollback to 1`.
- `-f values-prod.yaml` : seules les différences ; `--dry-run` rend sans appliquer.
- Kustomize : `kubectl kustomize` montre, `apply -k` applique, overlay = namespace + replicas + images + patches + labels.
- Registry : `--digestfile` = digest du registry = `imageID` sur le nœud ; `image@sha256:` dans le Deployment.
- `helm uninstall` garde la PVC ; `k3s-uninstall.sh` retire tout.